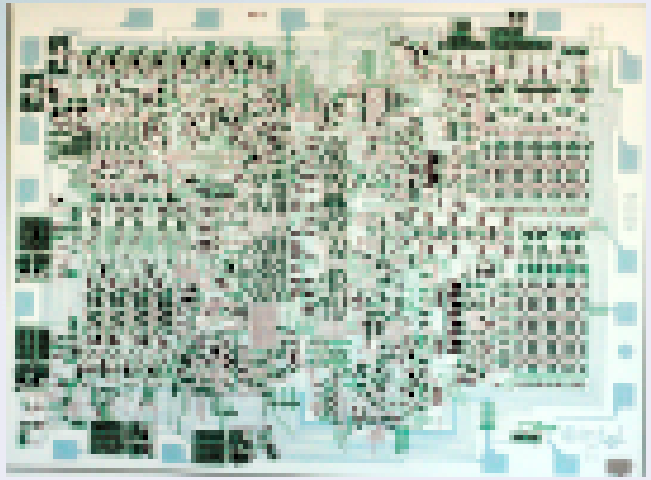

Lecture 7: stack.asm walkthrough

CSCI11: **Computer Architecture and Organization**

Review of stack.asm

1. Review stack_lib code starting with INIT_STK
2. Review stack_lib, constants
3. Review stack_lib, POP, PUSH, and FAIL
4. Review application code, to swap two variables

stack_lib: documentation

```
; stack_lib:
; *************** STACK LIBRARY CODE ************************************
; Library file, use for stack push and pop
; Requires additional code, stack functionality only

; Usage:
;   1. JSR STK_INIT - setup stack, must be performed
;   2. JSR PUSH - R0 will be pushed onto stack, Z flag -> no errors
;   3. JSR POP - R0 will return w/ top of stack, Z flag -> no errors
```

```
; Registers:
; R0: Returns pop value, receives push value
; R1: temp register, saved and restored
; R4: Return status, non-zero => error
; R6: Stack pointer

; Memory:
; USER_STACK - top of stack (TOS)
; STACK_SIZE - number of locations in stack
; EMPTY: (- USER_STACK) => stack empty
; FULL: -( USER_STACK + STACK_SIZE) => stack full
; SAVE_R1: x0000: Save register R1
```

stack_lib: *STK_INIT* setup stack pointer and EMPTY

```
STK_INIT        ; Stack initialization, call prior to using stack
                ; R1 is temp register, save it
                ; Initialize stack pointer (TOS)
                ; Change sign of stack pointer and save as EMPTY
                ST R1, SAVE_R1
                LD R6,USER_STACK    ; setup stack
                LD R1, USER_STACK
                NOT R1,R1
                ADD R1,R1,#1
                ST R1, EMPTY        ; save –TOS as EMPTY
```

stack_lib: *STK_INIT* setup FULL

```
; get size of stack (number of entries)
; calculate last stack memory location
; change sign and save as FULL
LD R1, STACK_SIZE
NOT R1,R1       ;
ADD R1,R1,#1    ;
ADD R1, R6, R1  ; last stack location
NOT R1, R1      ; change sign and save
ADD R1, R1, #1  ;
ST R1, FULL     ; change –last location as FULL
RET
```

stack_lib: constants

```
                ; stack data
USER_STACK      .FILL x4000     ; Top of stack
STACK_SIZE      .FILL x04       ; number of entries in stack, used to calc FULL
FULL            .FILL x0000     ; calculated, negative of last legal entry
EMPTY           .FILL x0000     ; calculated, negative top of stack
SAVE_R1         .FILL x0000     ; Used to save and restore R1
ERR_STACK       .STRINGZ "ERROR: Stack full or empty"
NORMAL          .STRINGZ "Normal Termination"
; *************** END OF STACK LIBRARY CODE ***************************************

                .END
```

stack_lib: *PUSH*

```
PUSH            ; put R0 on top of stack
                ; R1 is temp register, save it
                ; R4 contains condition code, non-zero is error
                ST R1,SAVE_R1   ; save R1, use as temp reg
                LD R1, FULL     ; check for full stack
                ADD R1,R6,R1    ;
                BRz FAIL        ; Branch if stack is full

                ADD R6,R6,#-1   ; Decrement
                STR R0,R6, #0   ; and store (PUSH)
                BRnzp SUCCESS
```

stack_lib: *POP*

```
POP             ; POP - return top of stack in R0
                ; R1 is temp register, save it
                ; R4 contains condition code, non-zero is error
                ST R1,SAVE_R1
                LD R1,EMPTY
                ADD R1,R6,R1
                BRz FAIL

                LDR R0,R6, #0   ; Load
                ADD R6,R6, #1   ; and increment (POP)
SUCCESS         LD R1,SAVE_R1   ; Restore original
                AND R4,R4,#0    ; R4 <-- success
                RET
```

stack_lib: *FAIL*

```
FAIL            ; Common failure code for POP and PUSH
                LD R1,SAVE_R1   ; Restore original
                AND R4,R4,#0    ; ensure R4 is zero
                ADD R4,R4,#1    ; RS <-- failure
                RET
```

application documentation - registers and memory

```
; stack:
; Demonstrates:
; Using stack_lib for stack functionality
; Uses stack to swap values in two memory locations
; Uses non-zero flag for error code (industry standard)
; Uses subroutines:  STK_INIT, PUSH and POP

; Usage:
; Set up two values then swap them using stack

; Registers:
; R0: Returns pop value, receives push value
; R3: Addresses of values

; Memory:
; A: xAAAA: Value to be pushed onto stack
; B: xBBBB: Value to be pushed onto stack
```

application: setup

```
                .ORIG x3000
                BR      START
                ; User program data
A               .FILL xAAAA     ; Value to be pushed onto stack
B               .FILL xBBBB     ; Value to be popped from stack
```

application: PUSH to save values on stack

```
START
                JSR STK_INIT    ; Initialize stack, REQUIRED

                ; Set up values to be swapped and push onto stack
                LD R0,A         ; Load a value to be pushed onto stack
                JSR PUSH        ; Push a value onto the stack
                BRnp ERROR      ; Branch if stack error
                LD R0,B         ; Load second value to be pushed onto stack
                JSR PUSH        ; Push a value onto the stack
                BRnp ERROR      ; Branch if stack error
```

application: error checking OVERFLOW

```
                ; Uncomment block to show overflow error
                ; Code attempts to push 5 values onto stack
                ; JSR PUSH      ; Extra push onto stack (SAFE)
                ; BRnp ERROR    ; Branch if stack error
                ; JSR PUSH      ; Extra push onto stack (SAFE)
                ; BRnp ERROR    ; Branch if stack error
                ; JSR PUSH      ; Extra push onto stack (OVERFLOW)
                ; BRnp ERROR    ; Branch if stack error
```

application: error checking UNDERFLOW

```
                ; Uncomment block to show underflow error
                ; Code attempts an extra pop from stack
                ; JSR POP       ; Extra pop from stack (UNDERFLOW)
                ; BRnp ERROR    ; Branch if stack error
```

application: POP to swap values

```
; Pop values and swap locations
LEA R3, A        ; Load address of A
JSR POP          ; Pop B from stack
BRnp ERROR       ; Branch if stack error
STR R0, R3, #0   ; Store B in A

LEA R3, B        ; Load address of B
JSR POP          ; Pop A from stack
BRnp ERROR       ; Branch if stack error
STR R0, R3, #0   ; Store A in B
BR NORM          ; Done
```

application: error handling

```
                ; print error message and exit
ERROR           LEA R0, ERR_STACK
                PUTS
                BR EXIT

                ; normal termination
NORM            LEA R0, NORMAL
                PUTS
EXIT            HALT
```